EC-231 Operating Systems

BS(CE)-2021

LAB REPORT # 10

Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz
21-CE-020	Muhammad Ibthesam
21-CE-021	Yawar Ali Malik

Department of Computer Engineering

BS Computer Engineering Program

HITEC University Taxila

	Presentation (Format (0.5) + Title (0.5) + Conclusion(1) + Procedure(2) + Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

_____

Lab Instructor.
Engr. Fasih Ahmad

Experiment #10

Implementation of First Come First Served and Shortest Job First Scheduling Algorithm

Objective:

In this lab we will learn about FCFS (First Come First Served) Scheduling Algorithm.

Software Used:

1.Virtual box
2.Ubuntu
3.Sublime text editor
4.GCC COMPLIER

Procedure:

FCFS: CPU scheduler will decide which process should be given the CPU for its execution. For this it uses different algorithm to choose among the process. One among that algorithm is First Come First Serve (FCFS) algorithm. First come first serve (FCFS) scheduling algorithm simply schedules the jobs according to their arrival time. The process that enters the ready queue first is scheduled first, regardless of the size of its next CPU burst. The lesser the arrival time of the job, the sooner will the job get the CPU. FCFS scheduling may cause the problem of starvation if the burst time of the first process is the longest among all the jobs.

• Non-preemptive. • Ready queue is a FIFO queue. • Jobs arriving are placed at the end of queue. • Dispatcher selects first job in queue and this job runs to completion of CPU burst.

• Advantages: simple, low overhead. • Disadvantage: inappropriate for interactive systems. Given n processes with their burst times and arrival times, the task is to find average waiting time and an average turnaround time using FCFS scheduling algorithm. FIFO simply queues processes in the order they arrive in the ready queue. Here, the process that comes first will be executed first and next process will start only after the previous gets fully executed.

Completion Time: Time at which the process completes its execution.

Turn Around Time: Time Difference between completion time and arrival time.

Turn Around Time = Completion Time – Arrival Time

Waiting Time (W.T): Time Difference between turnaround time and bursttime.

Waiting Time = Turn Around Time – Burst Time.

Algorithm:

1. Input the processes along with their burst time (bt) and arrival time(at).

2. Find waiting time (wt) for all processes.
3. As first process that comes need not to wait so waiting time for process 1 will be 0 i.e. wt[0] =0.
4. Find waiting time for all other processes i.e., for a given process i:wt[i] = (bt[0] + bt[1] +..... bt[i-1]) - at[i].
5. Find turnaround time = waiting_time + burst_time for all processes.
6. Find average waiting time = total_waiting_time / no_of_processes.
7. Similarly, find average turnaround time = total_turn_around_time / no_of_processes.
8. Compute Average Waiting Time of processes.
9. Compute Average Turnaround Time of processes.
10. Display the Gantt chart.

SJF Algorithm:

1. Input the processes along with their burst time (bt).
2. The process which have shortest burst will execute first i.e. sort processes according to their burst time.
3. If more than 1 process has same burst time then FCFS algorithm will be followed.
4. Find waiting time (wt) for all processes.
5. As first process that comes need not to wait so waiting time for process 1 will be 0 i.e. wt[0] = 0.
6. Find waiting time for all other processes i.e. for process i,wt[i] = bt[i-1] + wt[i-1] .
7. Find turnaround time = waiting_time + burst_time for all processes.
8. Find average waiting time = total_waiting_time / no_of_processes.
9. Similarly, find average turnaround time = total_turn_around_time / no_of_processes.
10. Compute Average Waiting Time of processes.
11. Compute Average Turnaround Time of processes.
12. Display the Gantt chart.

Tasks

Task 1:

Implement First Come First Serve CPU Scheduling Algorithm.

CODE:

```
#include <stdio.h>

#define MAX_PROCESS 10

typedef struct {

    int pid;

    int arrival_time;

    int burst_time;

    int remaining_time;
```

```
    int waiting_time;
    int turnaround_time;
} Process;
void fcfs(Process processes[], int n) {
    int current_time = 0;
    int completed = 0;
    int total_waiting_time = 0;
    int total_turnaround_time = 0;
    while (completed < n) {
        int min_index = -1;


        for (int i = 0; i < n; i++) {
            if (processes[i].arrival_time == current_time &&
processes[i].remaining_time > 0) {
                min_index = i;
                break;
            }
        }
        if (min_index == -1) {
            current_time++;
            continue;
        }
        processes[min_index].remaining_time--;
        current_time++;
        if (processes[min_index].remaining_time == 0) {
            completed++;
```

```
            processes[min_index].waiting_time = current_time - processes[min_index].arrival_time - processes[min_index].burst_time;
            processes[min_index].turnaround_time = current_time - processes[min_index].arrival_time;
            total_waiting_time += processes[min_index].waiting_time;
            total_turnaround_time += processes[min_index].turnaround_time;
        }
    }
    double average_waiting_time = (double)total_waiting_time / n;
    double average_turnaround_time = (double)total_turnaround_time / n;
    printf("-GANTT CHART-\n");
    for (int i = 0; i < n; i++) {
        printf("p%d ", processes[i].pid);
    }
    printf("\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < processes[i].burst_time; j++) {
            printf("I ");
        }
        printf("\n");
    }
    printf("Average Waiting Time of Processes : %.4f\n", average_waiting_time);
    printf("Average Turnaround Time of Processes : %.4f\n", average_turnaround_time);
}
int main() {
    Process processes[MAX_PROCESS];
```

```
    int n;
    printf("Enter the number of processes : ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter Arrival Time for process p%d : ", i);
        scanf("%d", &processes[i].arrival_time);
        printf("Enter Burst Time for process p%d : ", i);
        scanf("%d", &processes[i].burst_time);
        processes[i].pid = i;
        processes[i].remaining_time = processes[i].burst_time;
        processes[i].waiting_time = 0;
        processes[i].turnaround_time = 0;
    }
    fcfs(processes, n);
    return 0;
}
```

FILE:

```
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ subl fcfs.c
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ gcc fcfs.c -o fcfs
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ ./fcfs
```

OUTPUT:

```
Enter the number of processes: 3

Enter arrival time and burst time for process 1: 0  24

Enter arrival time and burst time for process 2: 0 3

Enter arrival time and burst time for process 3: 0 3

Process Arrival Time    Burst Time      Waiting Time    Turnaround Time
P1      0               24              0               24
P2      0               3               24              27
P3      0               3               27              30
Average waiting time = 17.00
Average turnaround time = 27.00
```

Task 2:
Implement shortest Job First CPU Scheduling Algorithm.
CODE:

```
#include <stdio.h>
#define MAX_PROCESSES 10
typedef struct {
    int at;
    int bt;
    int wt;
    int tat;
} Process;
void fcfs(Process p[], int n) {
    int total_wt = 0, total_tat = 0;
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (p[j].at > p[j + 1].at) {
                Process temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
    p[0].wt = 0;
```

```
    p[0].tat = p[0].bt;
    for (i = 1; i < n; i++) {
        p[i].wt = p[i - 1].bt + p[i - 1].wt;
        p[i].tat = p[i].wt + p[i].bt;
        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }
    printf("Process Name | Burst Time (bt) | Arrival Time (at) | Waiting Time
(wt) | Turnaround Time |\n");
    for (i = 0; i < n; i++) {
        printf("%s%11d | %11d | %13d | %14d |\n", "p", p[i].bt, p[i].at, p[i].wt,
p[i].tat);
    }
    printf("\nAverage Waiting Time = %.2f\n", (float)total_wt / n);
    printf("Average Turnaround Time = %.2f\n", (float)total_tat / n);
    printf("\nGANTT CHART:\n");
    int time = 0;
    for (i = 0; i < n; i++) {
        for (j = p[i].at; j < p[i].at + p[i].bt; j++) {
            printf("%d ", j);
        }
        time = p[i].at + p[i].bt;
    }
    printf("\n");
}
int main() {
    Process p[MAX_PROCESSES];
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time and Burst Time for each process:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d%d", &p[i].at, &p[i].bt);
    }
    fcfs(p, n);
    return 0;
}
```

FILE:

```
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ subl fcfs.c
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ gcc fcfs.c -o fcfs
huzaifashahbaz@huzaifashahbaz-VirtualBox:~/huzaifa134$ ./fcfs
```

OUTPUT:

```
Enter the number of processes: 4
Enter Arrival Time and Burst Time for each process:
0 2
2 5
7 3
8 10
Process Name | Burst Time (bt) | Arrival Time (at) | Waiting Time (wt) | Turnaround Time |
p          2 |               0 |                 0 |                 2 |
p          5 |               2 |                 2 |                 7 |
p          3 |               7 |                 7 |                10 |
p         10 |               8 |                10 |                20 |

Average Waiting Time = 4.75
Average Turnaround Time = 9.25

GANTT CHART:
0 1 2 3 4 5 6 7 8 9 8 9 10 11 12 13 14 15 16 17
```

Conclusion:

In conclusion, Processes are executed in the order they arrive, with priority given to those based on arrival time by the First Come First Served (FCFS) scheduling algorithm. Although it's straightforward and quick to use, it might have lengthier average wait times, particularly for lengthy processes that arrive later. In order to reduce waiting times and maximize throughput, smallest Job First (SJF) scheduling chooses the process with the smallest burst time to execute next.